- `IgnoreSection(string section)`—Ignores an unrendered section. If a section is defined in a page but not rendered, the Razor Page throws an exception unless the section is ignored.
- `IgnoreBody()`—Ignores the unrendered body of the Razor Page. Layouts must call either `RenderBody()` or `IgnoreBody()`; otherwise, they will throw an `InvalidOperationException`.

Layout files and sections provide a lot of flexibility for building sophisticated UIs, but one of their most important uses is in reducing the duplication of code in your application. They're perfect for avoiding duplication of content that you'd need to write for every view. But what about those times when you find you want to reuse part of a view somewhere else? For those cases, you have partial views.

17.4.3 Using partial views to encapsulate markup

Partial views are exactly what they sound like: part of a view. They provide a means of breaking up a larger view into smaller, reusable chunks. This can be useful for both reducing the complexity in a large view by splitting it into multiple partial views or for allowing you to reuse part of a view inside another.

Most web frameworks that use server-side rendering have this capability. Ruby on Rails has partial views, Django has inclusion tags, and Zend has partials. These all work in the same way, extracting common code into small, reusable templates. Even client-side templating engines such as Mustache and Handlebars, used by client-side frameworks like Angular and Ember, have similar "partial view" concepts.